

Le mode d'emploi du développeur

Lancer l'app, enregistrer une modification, publier une mise à jour, gérer les captures d'écran. Chaque démarche se fait en quelques commandes, et ce guide te dit lesquelles, dans quel ordre, et pourquoi.

Les repères

Le projet	D:\photogramme
Les scripts	D:\photogramme\scripts
La page GitHub	github.com/ArnGui/photogramme
Le lien à partager	github.com/ArnGui/photogramme/releases/latest

Sommaire

- 0 Une seule fois : préparer ta machine
- 1 Ouvrir PowerShell au bon endroit
- 2 Lancer Photogramme
- 3 Enregistrer une modification
- 4 Publier une mise à jour
- 5 Les captures d'écran et la page GitHub
- 6 Mettre à jour FFmpeg
- 7 Quand ça coince
- 8 Aide-mémoire

Comment lire ce guide. Les blocs gris sont des commandes à taper dans PowerShell, **une ligne à la fois**, en attendant que la précédente soit terminée (le curseur revient après PS D:\photogramme>). Les encadrés orange signalent un piège, les gris une astuce.

0

Une seule fois : préparer ta machine

Trois réglages à faire une fois pour toutes. Ensuite, toutes les commandes du guide sont courtes.

1 Ranger les scripts dans le projet

Télécharge les dernières versions de `prepare-release.ps1` et `publish-public.ps1` envoyées par Claude, puis :

```
cd D:\photogramme
Copy-Item "$HOME\Downloads\publish-public.ps1" scripts\ -Force
Copy-Item "$HOME\Downloads\prepare-release.ps1" scripts\ -Force
```

Pourquoi : rangés dans le projet, ils sont sauvegardés sur GitHub avec le reste et tu n'as plus à les chercher. Ils trouvent tout seuls le dossier du projet. Le dossier `scripts` contient déjà `setup-ffmpeg.ps1` et `run-tests.ps1`.

2 Autoriser tes propres scripts

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
Unblock-File scripts\*.ps1
```

Pourquoi : par défaut, Windows refuse d'exécuter le moindre script PowerShell. `RemoteSigned` autorise ceux qui sont sur ton disque et continue de bloquer ceux qui viennent d'Internet sans signature. `Unblock-File` retire le marquage « téléchargé » de tes scripts. Si une confirmation s'affiche, réponds 0.

Plan B. Si Windows refuse de changer ce réglage, utilise la forme longue partout où le guide écrit `.\scripts\nom.ps1 : powershell -ExecutionPolicy Bypass -File scripts\nom.ps1`, suivi des mêmes options.

3 Faire le ménage

Dans tes Téléchargements, supprime `fix-preview.ps1`, `publish-github.ps1` et les anciennes copies des deux scripts. Ils ont servi une fois et ne doivent plus être lancés.

1

Ouvrir PowerShell au bon endroit

Le plus simple : dans l'Explorateur, ouvre `D:\photogramme`, clique dans la barre d'adresse, tape `powershell` et appuie sur Entrée. PowerShell s'ouvre directement dans le projet. Depuis une fenêtre déjà ouverte :

```
cd D:\photogramme
```

Toutes les commandes de ce guide partent de là : le début de ligne doit afficher `PS D:\photogramme>`.

Inutile. Pas besoin d'ouvrir PowerShell « en tant qu'administrateur ». C'est même déconseillé : une fausse manipulation y a plus de conséquences.

« git n'est pas reconnu » (ou gh, cargo, npm)

Une fenêtre PowerShell lit la liste des programmes installés à son ouverture. Après une installation, ferme-la et rouvre-la, ou tape ces deux lignes :

```
$m = [Environment]::GetEnvironmentVariable('Path','Machine')
$env:Path = $m + ';' + [Environment]::GetEnvironmentVariable('Path','User')
```

2

Lancer Photogramme

Pour...	Comment
L'utiliser au quotidien	Menu Démarrer > Photogramme. C'est la version installée.
Tester une modification	<code>npm run tauri dev</code> L'app se recharge quand le code change. <code>Ctrl + C</code> pour l'arrêter.
Mettre à jour ta version installée, sans rien publier	Les deux commandes ci-dessous.

```
npm run tauri build
explorer src-tauri\target\release\bundle\nsis
```

Double-clique ensuite sur le fichier `Photogramme_..._x64-setup.exe`. L'installation par-dessus l'ancienne garde tes réglages et tes préférences d'overlay.

Patience. La compilation prend de 3 à 6 minutes, et la ligne `Building [====> ] 435/437` peut rester figée plusieurs minutes : c'est l'optimisation finale, elle travaille. Le Gestionnaire des tâches (`Ctrl + Maj + Échap`) doit montrer `rustc.exe` ou `link.exe` en activité.

3

Enregistrer une modification

Après chaque changement qui marche, même petit, fait avec Claude ou à la main.

1 Vérifier

```
npx tsc --noEmit
.\scripts\run-tests.ps1
npm run tauri dev
```

`tsc` : aucune sortie veut dire aucune erreur de type. `run-tests` : les tests Rust et TypeScript, avec le vrai FFmpeg. `tauri dev` : essaie toi-même la fonction modifiée. Les tests ne remplacent pas ton œil.

2 Sauvegarder sur GitHub

```
git status
git add -A
git commit -m "Fix black bars toggle scrolling the window"
git push
```

Commande	Ce qu'elle fait
<code>git status</code>	Liste les fichiers modifiés. Regarde qu'il n'y a pas d'intrus (un film, un fichier perso).
<code>git add -A</code>	Prépare tous ces changements.
<code>git commit -m "..."</code>	Crée un point de retour, avec une phrase qui dit ce qui change. En anglais, puisque le dépôt est public.
<code>git push</code>	Envoie le tout sur GitHub.

À savoir. Le dépôt est public : un push est visible tout de suite par tout le monde. Ce n'est pas pour autant une mise à jour pour les utilisateurs : ils ne reçoivent rien tant que tu ne publies pas une nouvelle version (section 4).

Revenir en arrière

Pour...	Commande
Voir les derniers points de retour	<code>git log --oneline -10</code>
Annuler tout ce qui n'est pas encore commité	<code>git restore .</code> Irréversible : efface les changements non enregistrés.
Revenir sur un commit déjà envoyé	Demande à Claude : la bonne commande dépend de la situation.

4

Publier une mise à jour

Quand tu as un ensemble de changements testés que tu veux donner aux utilisateurs.

1 Choisir le numéro

Ce qui change	Exemple
Une correction de bug, rien de nouveau	0.2.0 → 0.2.1
Une ou plusieurs nouvelles fonctions	0.2.1 → 0.3.0
La version que tu considères aboutie et stable	0.9.0 → 1.0.0

C'est la convention Semantic Versioning (semver.org), celle qu'attendent les développeurs qui liront ta page.

2 Tester la version installée

Section 2, dernier cas : construis, installe, ouvre un film, fais une capture et un export en lot. C'est la seule façon de voir ce que verront les autres : l'installateur, FFmpeg embarqué, les licences.

3 Publier

```
.\scripts\publish-public.ps1 -Version 0.3.0 -Notes "Faster shot detection. PNG export."
```

Le texte de `-Notes` s'affiche en tête de la page de la version : une ou deux phrases, en anglais, du point de vue de l'utilisateur (ce qui change pour lui, pas dans le code).

Le script travaille seul pendant 10 à 15 minutes :

- il écrit le nouveau numéro dans les quatre fichiers qui le portent (`tauri.conf.json`, `package.json` et les deux `Cargo.toml`) ;
- il régénère les licences des dépendances et la page GitHub ;
- il vérifie le code, construit l'installateur et calcule son empreinte SHA-256 ;
- il enregistre et envoie le tout sur GitHub ;
- il crée la page de la version avec l'installateur, tes notes et l'empreinte.

4 Vérifier

Ouvre le lien à partager dans une fenêtre de navigation privée : la nouvelle version doit apparaître, avec son installateur.

En cas d'arrêt. Si le script s'arrête en rouge, lis le message : il dit quoi faire. Une fois le problème réglé, relance exactement la même commande : les étapes déjà faites sont reconnues. S'il te dit `Release v0.3.0 already exists`, c'est que tu as oublié de changer le numéro, ou que cette version est déjà publiée : réponds `N`.

5

Les captures d'écran et la page GitHub

Emplacement	Rôle
<code>docs\screenshot.jpg</code>	La grande image en haut de la page. Une seule.
<code>docs\screenshots\</code>	Toutes les autres, dans une section « Screenshots ». Autant que tu veux.

Dans docs\screenshots, **le nom du fichier fait tout** : le numéro donne l'ordre, le reste devient la légende. Nomme-les en anglais, comme la page.

Nom du fichier	Ce qui s'affiche
01-shot-list.jpg	1re image, légende « Shot list »
02-overlay-preview.jpg	2e image, légende « Overlay preview »
03-color-barcode.png	3e image, légende « Color barcode »

```
explorer docs
.\scripts\publish-public.ps1 -SkipRelease
```

-SkipRelease : met à jour la page et les licences, enregistre et envoie, sans reconstruire l'installateur ni créer de version. Environ une minute. Pour retirer une capture, supprime le fichier et relance la même commande.

Une bonne capture

- Montre l'app au travail : un film ouvert, la liste des plans, l'aperçu d'export avec la bande de palette.
- JPEG d'environ 1600 px de large, qualité 85 : net sur un écran haute densité, sous 500 Ko.
- Le dépôt est public : un plan dont tu as les droits, sans personne reconnaissable sans son accord.

Ne modifie pas la page à la main. La page GitHub (README.md), LICENSE et les deux fichiers THIRD_PARTY_... sont **fabriqués par les scripts** à chaque lancement. Si tu modifies README.md à la main, ou sur le site GitHub, tes changements seront effacés au prochain lancement. Pour changer le texte de la page, demande à Claude de modifier le modèle dans publish-public.ps1.

6

Mettre à jour FFmpeg

Rare. Seulement si une nouvelle version de FFmpeg corrige un problème qui te concerne.

```
.\scripts\setup-ffmpeg.ps1 -Version 8.2
.\scripts\prepare-release.ps1
```

La première ligne télécharge la version LGPL de FFmpeg dans src-tauri\binaries (remplace 8.2 par le numéro voulu). À la fin, cuda doit apparaître dans la liste affichée : c'est ce qui permet le décodage par la carte NVIDIA. La seconde réécrit THIRD_PARTY_NOTICES.txt avec la nouvelle version et le lien vers son code source, et télécharge l'archive source à joindre à la prochaine version. Elle ne touche plus à la page GitHub.

Ensuite : teste (section 2), puis publie une mise à jour (section 4) avec une note du genre "Updated FFmpeg to 8.2."

Piège. Ne remplace jamais ffmpeg.exe à la main par une version « full » ou « GPL » trouvée ailleurs : elle ne pourrait pas être distribuée avec Photogramme dans les conditions actuelles. prepare-release.ps1 refuse de continuer si c'est le cas.

7

Quand ça coince

Ce que tu vois	Ce qu'il faut faire
« ... n'est pas reconnu » (git, gh, cargo, npm)	Rouvre PowerShell, ou tape les deux lignes de la section 1.
« l'exécution de scripts est désactivée sur ce système »	Section 0, étape 2.
Le script est introuvable	La ligne doit commencer par PS D:\photogramme>. Sinon : cd D:\photogramme.
La compilation semble bloquée	Normal pendant 2 à 6 minutes (section 2, encadré « Patience »).
Une erreur TypeScript	Ne commite pas. Envoie le message à Claude.
failed to satisfy license requirements	Une nouvelle dépendance a une licence inconnue de la liste blanche. Envoie le message à Claude, qui vérifiera sa compatibilité avec la GPL avant de l'ajouter à about.toml.
Push refusé (rejected, fetch first)	Quelque chose a été modifié directement sur le site GitHub. Tape git pull, puis git push.
Release ... already exists	Numéro de version oublié : relance avec -Version.
Un utilisateur a un avertissement Windows ou antivirus	Normal pour un installateur non signé. Renvoie-le vers l'empreinte SHA-256 des notes de version.
Autre chose en rouge	Rien n'est cassé tant que tu n'as rien supprimé. Copie le message entier et envoie-le à Claude.

8

Aide-mémoire

Toutes les commandes partent de PS D:\photogramme>.

Je veux...	Je tape...
Utiliser l'app	Menu Démarrer > Photogramme
Tester une modification	<code>npm run tauri dev</code>
Vérifier le code	<code>npx tsc --noEmit</code>
Lancer les tests	<code>.\scripts\run-tests.ps1</code>
Voir ce qui a changé	<code>git status</code>
Sauvegarder sur GitHub	<code>git add -A</code> <code>git commit -m "..."</code> <code>git push</code>
Mettre à jour ma version installée	<code>npm run tauri build</code> puis l'installateur dans <code>src-tauri\target\release\bundle\nsis</code>
Publier une mise à jour	<code>.\scripts\publish-public.ps1 -Version 0.3.0 -Notes "..."</code>
Mettre à jour la page ou les captures	<code>.\scripts\publish-public.ps1 -SkipRelease</code>
Mettre à jour FFmpeg	<code>.\scripts\setup-ffmpeg.ps1 -Version x.y</code> <code>.\scripts\prepare-release.ps1</code>
Ouvrir la page GitHub	<code>gh repo view --web</code>
Lire les retours des utilisateurs	Onglet « Issues » de la page GitHub

Astuce. Garde le bouton « Watch » activé en haut de ta page GitHub : tu reçois un mail à chaque nouveau ticket, et les premiers jours après une version sont ceux où tombent les bugs sur des configurations différentes de la tienne.